SpecFirst React A11y

Phase 7 Documentation: Bob Patches Component

How the locked contract and baseline failures are assembled into a constrained IBM Bob remediation task

---

Phase 7 is where IBM Bob earns its place in the pipeline. The previous six phases built a tamper-evident contract and proved it fails. Phase 7 gives Bob a precise, bounded task: here are the failing checks, here is the frozen contract, patch only this component file, and do not touch anything else.

Prepared for the IBM Bob Hackathon workflow

1. Phase 7 in one sentence

Phase 7 reads the baseline failures and the locked spec, generates a constrained Bob remediation prompt, waits for Bob to patch the target component in IBM Bob IDE, and then runs a patch scope guard to confirm Bob did not edit any protected artifact.

Pipeline context:

```
Phase 6: Run baseline test Phase 7: Bob patches component ← you are
here Phase 8: Verify and report
```

The central design rule is that Bob patches against a contract Bob did not create. SpecFirst constrains. Bob patches. SpecFirst verifies. Phase 7 is the handoff to Bob and the guard on the way back.

2. Purpose

- Translate baseline failures and locked-spec requirements into a precise Bob remediation task.
- Generate bobPrompt.md — a deterministic, constrained prompt that names the exact failing checks, forbidden files, focus management strategy, and patch constraints from the manifest.
- After Bob has patched, verify that Bob only modified the allowed target component and did not touch the locked spec, generated tests, or harness.
- Verify immutable artifact hashes have not changed since Phase 5.
- Record the Bob session export path and write bobPatchResult.json.

3. Inputs

Input artifact	Role in Phase 7
baselineResult.json	Must have status red-confirmed. Provides failing check IDs, failure messages, and artifact hash snapshots from Phase 6.
lockedSpec.json	Source of component path, Bob patch constraints, focus management strategy, manual review boundaries, and accessible name binding.
testGenerationResult.json	Provides test file path, harness file path, and baseline command for inclusion in the Bob prompt and the patch scope guard.
Bob session export (optional)	The markdown export from IBM Bob IDE's task history. Required for complete hackathon evidence. If absent, bobPatchResult status becomes bob-session-missing rather than patched.

4. Outputs

Output artifact	Purpose
bobPrompt.md	The constrained remediation prompt for IBM Bob IDE. Generated deterministically from locked spec plus baseline failures. Bob reads this. Bob does not write it.
bobPatchResult.json	Machine-readable record of the Phase 7 outcome. Status, component identity, inputs, Bob session reference, patch scope result, and artifact integrity. Primary input to Phase 8.

5. Scope boundaries

Phase 7 should do:

- Read baselineResult.json and refuse to proceed if status is not red-confirmed.
- Read lockedSpec.json to extract component path, Bob constraints, and focus management strategy.
- Generate bobPrompt.md deterministically from the above inputs.
- After Bob's patch, re-verify SHA-256 hashes of lockedSpec.json, the test file, and the harness file.

- Run git diff (unstaged + staged + committed) to identify which files changed.
- Compare changed files against the allowed list (target component only) and forbidden list (locked spec, tests, harness).
- Record Bob session export path if provided.
- Write bobPatchResult.json.

Phase 7 should not do:

- Decide whether the final tests will pass. That is Phase 8.
- Run Playwright tests. That is Phase 8.
- Edit lockedSpec.json, the generated test file, or the generated harness.
- Generate new tests or modify the locked spec checks.
- Claim accessibility compliance.
- Automatically retry if Bob's patch scope fails. A human must intervene.

6. Bob prompt structure

bobPrompt.md is generated deterministically from the pipeline state. It must be precise enough for Bob to act without additional context. The prompt includes:

Section	Content
Target component	Exact file path of the component Bob must patch. Only this file is allowed to change.
Locked spec reference	Path to lockedSpec.json and its integrity hash. Bob may read it but must not edit it.
Generated test reference	Path to the test file. Bob must not edit this file.
Harness reference	Path to the harness file. Bob must not edit this file.
Baseline failures	Structured list of failing check IDs and their failure messages, taken from baselineResult.failedChecks.
Patch constraints	The bobPatchConstraints array from lockedSpec.json: allowed ARIA attributes, allowed keyboard handlers, forbidden structural changes.
Focus management strategy	Explains aria-activedescendant: DOM focus stays on the trigger, the active option is referenced by aria-activedescendant, and aria-activedescendant must point to an existing option element ID.
Desired output	Bob summarizes files changed, checks addressed, props preserved, and assumptions made.

7. Patch scope guard

After Bob patches the target component, the patch scope guard runs automatically:

1. Collect changed files from `git diff --name-only` (committed), `git diff --name-only` (unstaged), and `git diff --name-only --cached` (staged).
2. Check each changed file against the forbidden list: lockedSpec.json, the test file, and the harness file.
3. If any forbidden file appears in the changed set, mark patchScope.status as failed.
4. Even if patchScope passes, re-verify artifact hashes before writing the result.

> The allowed changed files list contains only the target component path. Any change outside that list is not automatically a failure — there may be legitimate adjacent changes — but forbidden file changes always produce patch-scope-failed regardless of other changes.

8. Artifact integrity re-verification

Phase 7 re-hashes three artifacts after Bob's patch and compares them to the Phase 6 baseline record:

Artifact	Expected hash source	On mismatch
lockedSpec.json	baselineResult.lockedSpecHash	status: invalidated
Generated test file	baselineResult.testFileHashBeforeRun	status: invalidated
Generated harness file	baselineResult.harnessFileHashBeforeRun	status: invalidated

If any of these hashes changed, the pipeline cannot be trusted. Phase 8 will refuse to verify against an invalidated chain of custody.

9. Bob session evidence

For hackathon submission, each team member must export their Bob IDE task session and upload it to `bob_sessions/`. Phase 7 records the session export path in bobPatchResult.json. If the session is missing:

- bobPatchResult.status becomes bob-session-missing instead of patched.
- nextPhase.canContinue remains true so Phase 8 can still run.
- A warning is included: evidence is incomplete for hackathon submission.
- Phase 8 evidence report will clearly mark Bob session evidence as missing.

10. Status model

Status	Meaning	nextPhase
prompt-generated	bobPrompt.md was written. Bob has not yet patched. Waiting for human to run Bob IDE.	canContinue false
patched	Bob patched the target component, patch scope passed, artifact integrity passed, Bob session export present.	canContinue true
bob-session-missing	Patch scope passed and integrity passed, but no Bob session export was provided. Evidence incomplete.	canContinue true (with warning)
patch-scope-failed	Bob edited one or more forbidden files (locked spec, test file, or harness).	canContinue false
invalidated	Immutable artifact hashes changed after Phase 5. Chain of custody broken.	canContinue false
skipped	Baseline was not red-confirmed. No Bob patch should happen.	canContinue false
failed	Missing input artifact, malformed JSON, or unexpected implementation error.	canContinue false

11. Failure cases

- baselineResult.json missing or status not red-confirmed.
- lockedSpec.json or testGenerationResult.json missing from the run directory.
- Any immutable artifact hash changed — produces invalidated, not failed.
- Bob edited the test file, harness, or locked spec — produces patch-scope-failed.
- Bob session export path provided but file does not exist — treat as bob-session-missing.

12. Definition of done

- baselineResult.json has status red-confirmed before generating the prompt.
- bobPrompt.md is generated and includes failing check IDs, forbidden file list, focus strategy, and patch constraints.
- After Bob's patch, artifact integrity is verified against Phase 6 hash records.
- Patch scope guard runs git diff and confirms no forbidden files changed.

- bobPatchResult.json is written with one of: patched, bob-session-missing, patch-scope-failed, or invalidated.
- nextPhase.canContinue is true only on patched or bob-session-missing.
- Phase 7 does not run Playwright, modify the locked spec or tests, or claim compliance.

13. Final implementation note

> Phase 7 is the most visible phase in the hackathon demo. The Bob prompt, the patch, and the scope guard are where the trust model becomes tangible: SpecFirst constrains, Bob patches, SpecFirst guards. The patch scope guard is not optional. Without it, there is no proof that Bob respected the boundaries the pipeline was designed to enforce.

End of Phase 7 documentation.